PART 5

Consistency & Concurrency

34 CAP Theorem
35 Eventual Consistency
36 Optimistic Locking
37 Pessimistic Locking
38 Distributed Locks
39 Race Conditions
40 Deadlocks
41 Memory Leaks
42 Garbage Collection
43 Thread Safety
44 Backpressure

#034

CAP Theorem

WHAT IT IS

During a network Partition, a distributed store must choose Consistency or Availability.

WHY IT MATTERS

Frames every trade-off in distributed data design.

EXAMPLE

A bank ledger chooses CP (reject requests rather than show wrong balances). A social feed or like counter chooses AP (always respond, reconcile later). Postgres primary = CP-leaning; Cassandra/DynamoDB = tunable AP-leaning.

#035

Eventual Consistency

WHAT IT IS

If no new updates arrive, all replicas eventually converge to the same value.

WHY IT MATTERS

Enables speed and availability at scale; the UI must tolerate stale reads.

EXAMPLE

```
dispatch(new IndexTaskInSearch($task))->afterCommit(
    ); // search catches up shortly after
```

#036

Optimistic Locking

WHAT IT IS

Assume conflicts are rare; detect them at save time using a version check.

WHY IT MATTERS

No locks held, high throughput; the loser retries or is told to refresh.

EXAMPLE

```
$updated = Task::where('id', $id)->where('version',
    $request->version)
    ->update(['title' => $request->title, 'version'
    => DB::raw('version + 1')]);

if (! $updated) abort(409, 'Task was changed by
    someone else. Reload and retry.');
```

#037

Pessimistic Locking

WHAT IT IS

Lock the row up front so nobody else can change it until you finish.

WHY IT MATTERS

Right choice when conflicts are likely (stock, balances).

EXAMPLE

```
DB::transaction(function () use ($id, $qty) {
    $product = Product::lockForUpdate()->findOrFail(
    $id);
    if ($product->stock < $qty) throw new OutOfStock;
    $product->decrement('stock', $qty);
});
```

#038

Distributed Locks

WHAT IT IS

A lock shared across servers (usually Redis) so only one process runs a critical section.

WHY IT MATTERS

Prevents duplicate cron runs, double processing, concurrent imports.

EXAMPLE

```
Cache::lock("import:{$account->id}", 300)->block(10,
    function () use ($account) {
    $this->runImport($account);
});
```

#039

Race Conditions

WHAT IT IS

Outcome depends on unlucky timing between concurrent operations.

WHY IT MATTERS

Causes oversold stock, double charges, lost updates; hard to reproduce.

EXAMPLE

```
// Bad: read-modify-write
$c = Counter::find(1); $c->value = $c->value + 1;
    $c->save();

// Good: atomic in the database
Counter::where('id', 1)->increment('value');

// Good: unique constraint + firstOrCreate to dedupe
```

#040

Deadlocks

WHAT IT IS

Two (or more) transactions each hold a lock the other needs, so none can proceed.

WHY IT MATTERS

The DB kills one transaction with an error; users see failures.

EXAMPLE

```
DB::transaction(fn () => $this->transfer($a, $b,
    $amount), attempts: 3); // retries on deadlock
```

#041

Memory Leaks

WHAT IT IS

Memory that is allocated and never released, so usage grows until the process dies.

WHY IT MATTERS

Long-running workers (queue, Octane, Reverb) are especially vulnerable.

EXAMPLE

```
Task::chunkById(1000, fn ($chunk) => $this->process(
    $chunk)); // not ->get() on 5M rows
```

#042

Garbage Collection

WHAT IT IS

Automatic reclaiming of memory no longer referenced.

WHY IT MATTERS

PHP uses reference counting plus a cycle collector; understanding it explains odd memory spikes.

EXAMPLE

```
gc_collect_cycles();                    // force a
    cycle collection
echo memory_get_usage(true) / 1048576;  // MB in use
unset($bigArray);                       // drop
    references so memory can be freed
```

#043

Thread Safety

WHAT IT IS

Code behaves correctly when run by several threads at once.

WHY IT MATTERS

Shared mutable state is the enemy. Classic PHP-FPM is share-nothing, but Octane/Swoole/FrankenPHP keep state between requests.

EXAMPLE

```
// Dangerous under Octane: a singleton holding
    per-request data
$this->app->singleton(CurrentTenant::class);   //
    leaks tenant across requests!

// Safe: scoped binding, reset every request
$this->app->scoped(CurrentTenant::class);
```

#044

Backpressure

WHAT IT IS

A mechanism for a slow consumer to tell producers to slow down.

WHY IT MATTERS

Prevents unbounded queues and out-of-memory crashes.

EXAMPLE

```
if (Queue::size('imports') > 10_000) {
    return response()->json(['message' => 'Busy, try
    later'], 503)->header('Retry-After', 60);
}
```

Found this useful?

Save it for later.

Share it with a teammate.

Full guide with all code examples:
the Backend Engineering Handbook (GitHub).